

ASSIGNMENT 8.4

SHAIK SABINA

2303A51303

BATCH – 05

TASK – 01

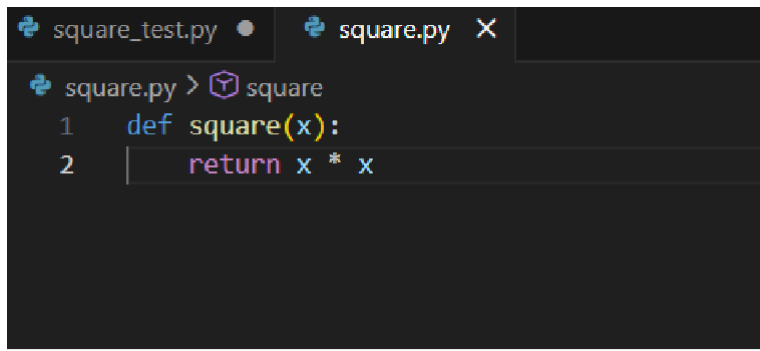
Prompt : Write Python unit tests using unit test for a square(x) function following TDD. After tests fail, implement square(x) in a separate file so all tests pass.

Code:

Square_test.py

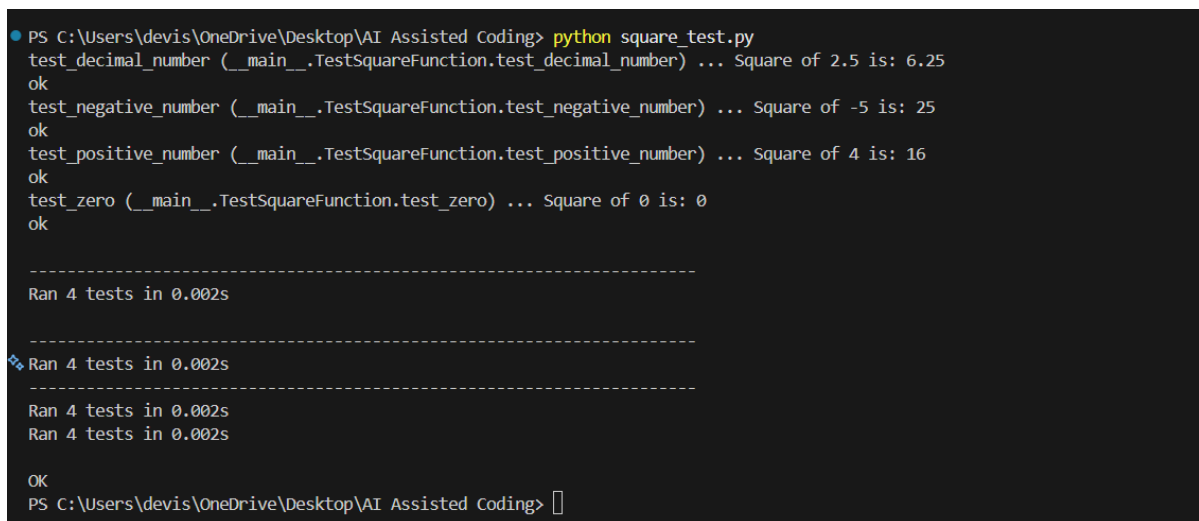
```
square_test.py > TestSquareFunction
1  '''Write Python unit tests using unittest for a square(x) function following TDD.
2  After tests fail, implement square(x) in a separate file so all tests pass.
3  '''
4  import unittest
5  from square import square
6  class TestSquareFunction(unittest.TestCase):
7
8      def test_positive_number(self):
9          result = square(4)
10         print("Square of 4 is:", result)
11         self.assertEqual(result, 16)
12     def test_zero(self):
13         result = square(0)
14         print("Square of 0 is:", result)
15         self.assertEqual(result, 0)
16
17     def test_negative_number(self):
18         result = square(-5)
19         print("Square of -5 is:", result)
20         self.assertEqual(result, 25)
21
22     def test_decimal_number(self):
23         result = square(2.5)
24         print("Square of 2.5 is:", result)
25         self.assertEqual(result, 6.25)
26 if __name__ == "__main__":
27     unittest.main(verbosity=2)
```

square.py



```
square_test.py  square.py X
square.py > square
1 def square(x):
2     return x * x
```

Output :



```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> python square_test.py
test_decimal_number (__main__.TestSquareFunction.test_decimal_number) ... Square of 2.5 is: 6.25
ok
test_negative_number (__main__.TestSquareFunction.test_negative_number) ... Square of -5 is: 25
ok
test_positive_number (__main__.TestSquareFunction.test_positive_number) ... Square of 4 is: 16
ok
test_zero (__main__.TestSquareFunction.test_zero) ... Square of 0 is: 0
ok

-----
Ran 4 tests in 0.002s

-----
❖ Ran 4 tests in 0.002s
-----
Ran 4 tests in 0.002s
Ran 4 tests in 0.002s

OK
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> 
```

Explanation :

- We follow Test-Driven Development (TDD), where tests are written first.
- The test file (test_square.py) contains unit tests using the unit test module.
- These tests check the square(x) function for:
 - positive numbers
 - zero
 - negative numbers
 - decimal values
- Initially, tests fail because the function is not implemented.
- Then the function square(x) is implemented in a separate file (square.py).
- After implementation, all test cases pass successfully.
- This ensures the function works correctly for different inputs.

TASK – 02

Prompt : #Write unit tests using unittest to validate email addresses for a user registration system. The tests should cover valid and invalid email formats such as missing @, missing domain, missing username, and incorrect structure. Then, implement the validate_email() function in a separate file so that all test cases pass. The implementation must strictly follow the behavior defined by the test cases.

Code :

test_email_validator.py

```
test_email_validator.py > TestEmailValidation > testMissing_atSymbol
1  #Write unit tests using unittest to validate email addresses for a user registration system.
2  #The tests should cover valid and invalid email formats such as missing @, missing domain, missing username, and incorrect structure.
3  #Then, implement the validate_email() function in a separate file so that all test cases pass.
4  #The implementation must strictly follow the behavior defined by the test cases.
5  from email_validator import validate_email
6
7  class TestEmailValidation(unittest.TestCase):
8
9      def test_valid_email(self):
10         self.assertTrue(validate_email("user@example.com"))
11
12     def test_missing_at_symbol(self):
13         self.assertFalse(validate_email("userexample.com"))
14
15     def test_missing_domain(self):
16         self.assertFalse(validate_email("user@"))
17
18     def test_missing_username(self):
19         self.assertFalse(validate_email("@example.com"))
20
21     def test_missing_dot_in_domain(self):
22         self.assertFalse(validate_email("user@example"))
23
24     if __name__ == "__main__":
25         unittest.main(verbosity=2)
26
```

email_validator.py

```
test_email_validation.py 2  email_validator.py X  app.log
email_validator.py > ...
1  def validate_email(email):
2      if "@" not in email:
3          return False
4
5      username, domain = email.split("@", 1)
6
7      if username == "" or domain == "":
8          return False
9
10     if "." not in domain:
11         return False
12
13     return True
14
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> python test_email_validation.py
test_missing_at_symbol (__main__.TestEmailValidation.test_missing_at_symbol) ... ok
test_missing_domain (__main__.TestEmailValidation.test_missing_domain) ... ok
test_missing_dot_in_domain (__main__.TestEmailValidation.test_missing_dot_in_domain) ... ok
test_missing_username (__main__.TestEmailValidation.test_missing_username) ... ok
test_valid_email (__main__.TestEmailValidation.test_valid_email) ... ok

-----
Ran 5 tests in 0.001s

OK
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> 
```

Explanation :

- Unit tests are written first to define valid and invalid email formats.
- Tests check cases like missing @, missing domain, and incorrect structure.
- The validate_email() function is implemented in a separate file.
- The function logic is written only to satisfy test expectations.
- This follows the Test-Driven Development (TDD) approach.

TASK – 03

Prompt : Write unit tests using unittest to determine the correct maximum value among three inputs. Test normal cases, equal values, negative numbers, and mixed values. After the tests are written, implement the find_max(a, b, c) function in a separate file so that all tests pass. The function logic must be derived strictly from the test cases.

Code :

```
max_value.py > ...
1  def find_max(a, b, c):
2      max_value = a
3
4      if b > max_value:
5          max_value = b
6      if c > max_value:
7          max_value = c
8
9      return max_value
10
```

test_max_value.py

```
test_max_value.py • max_value.py app.log
test_max_value.py > ...
1  #Write unit tests using unittest to determine the correct maximum value among three inputs.
2  #Test normal cases, equal values, negative numbers, and mixed values.
3  #After the tests are written, implement the find_max(a, b, c) function in a separate file so that all tests pass.
4  #The function logic must be derived strictly from the test cases."
5  import unittest
6  from max_value import find_max
7
8  class TestFindMax(unittest.TestCase):
9
10     def test_all_positive_numbers(self):
11         self.assertEqual(find_max(10, 20, 30), 30)
12
13     def test_all_negative_numbers(self):
14         self.assertEqual(find_max(-5, -2, -10), -2)
15
16     def test_mixed_numbers(self):
17         self.assertEqual(find_max(5, -3, 2), 5)
18
19     def test_all_equal_numbers(self):
20         self.assertEqual(find_max(7, 7, 7), 7)
21
22     def test_two_numbers_equal_and_max(self):
23         self.assertEqual(find_max(8, 8, 3), 8)
24
25 if __name__ == "__main__":
26     unittest.main(verbosity=2)
27
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> python test_max_value.py
test_all_equal_numbers (__main__.TestFindMax.test_all_equal_numbers) ... ok
test_all_negative_numbers (__main__.TestFindMax.test_all_negative_numbers) ... ok
test_all_positive_numbers (__main__.TestFindMax.test_all_positive_numbers) ... ok
test_mixed_numbers (__main__.TestFindMax.test_mixed_numbers) ... ok
test_two_numbers_equal_and_max (__main__.TestFindMax.test_two_numbers_equal_and_max) ... ok

-----
Ran 5 tests in 0.001s

OK
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> █
```

Explanation :

- Test cases are written first to define expected behavior.
- Tests cover positive numbers, negative numbers, equal values, and mixed cases.
- The function `find_max()` is implemented only after tests are completed.
- The logic is derived strictly from test expectations.
- This demonstrates the Test-Driven Development (TDD) approach.

TASK – 04

Prompt : Write unit tests using unittest for a ShoppingCart class.

The tests should verify adding items, removing items, and calculating total price. After all tests are written, implement the ShoppingCart class in a separate file so that all tests pass. The implementation must strictly follow the behavior defined in the test cases.

Code :

test_shopping_cart.py

```

test_shopping_cart.py > ...
1  #Write unit tests using unittest for a ShoppingCart class.
2  #The tests should verify adding items, removing items, and calculating total price.
3  #After all tests are written, implement the ShoppingCart class in a separate file so that all tests pass.
4  #The implementation must strictly follow the behavior defined in the test cases.
5  import unittest
6  from shopping_cart import ShoppingCart
7
8  class TestShoppingCart(unittest.TestCase):
9
10     def test_add_item(self):
11         cart = ShoppingCart()
12         cart.add_item("Book", 200)
13         self.assertEqual(cart.items["Book"], 200)
14
15     def test_remove_item(self):
16         cart = ShoppingCart()
17         cart.add_item("Pen", 20)
18         cart.remove_item("Pen")
19         self.assertNotIn("Pen", cart.items)
20
21     def test_calculate_total(self):
22         cart = ShoppingCart()
23         cart.add_item("Book", 200)
24         cart.add_item("Pen", 20)
25         self.assertEqual(cart.calculate_total(), 220)
26
27 if __name__ == "__main__":
28     unittest.main(verbosity=2)

```

Shopping_cart.py

```

shopping_cart.py > ...
1  class ShoppingCart:
2
3      def __init__(self):
4          self.items = {}
5
6      def add_item(self, name, price):
7          self.items[name] = price
8
9      def remove_item(self, name):
10         if name in self.items:
11             del self.items[name]
12
13     def calculate_total(self):
14         return sum(self.items.values())
15

```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> python test_shopping_cart.py
test_add_item (__main__.TestShoppingCart.test_add_item) ... ok
test_calculate_total (__main__.TestShoppingCart.test_calculate_total) ... ok
test_remove_item (__main__.TestShoppingCart.test_remove_item) ... ok

-----
Ran 3 tests in 0.001s

OK
❖ PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> 
```

Explanation :

- Unit tests define expected cart behavior.
- Tests cover adding, removing items, and total calculation.
- ShoppingCart class is implemented after tests.
- All logic is derived from test expectations.
- This demonstrates TDD in a class-based design.

TASK – 05

Prompt : Write unit tests using unittest to check whether a string is a palindrome. Tests should cover simple palindromes, non-palindromes, and case variations. After writing tests, implement the `is_palindrome()` function in a separate file so that all tests pass.

Code :

test_palindrome.py

```
test_palindrome.py > ...
1  #Write unit tests using unittest to check whether a string is a palindrome.
2  #Tests should cover simple palindromes, non-palindromes, and case variations.
3  #After writing tests, implement the is_palindrome() function in a separate file so that all tests pass.
4  import unittest
5  from palindrome import is_palindrome
6
7  class TestPalindrome(unittest.TestCase):
8
9      def test_simple_palindrome(self):
10         self.assertTrue(is_palindrome("madam"))
11
12     def test_not_palindrome(self):
13         self.assertFalse(is_palindrome("hello"))
14
15     def test_case_insensitive(self):
16         self.assertTrue(is_palindrome("RaceCar"))
17
18 if __name__ == "__main__":
19     unittest.main(verbosity=2)
20
```

palindrome.py

```
test_palindrome.py  palindrome.py X  app.log
palindrome.py > ...
1  def is_palindrome(text):
2      text = text.lower()
3      return text == text[::-1]
4
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> python test_palindrome.py
test_case_insensitive (__main__.TestPalindrome.test_case_insensitive) ... ok
test_not_palindrome (__main__.TestPalindrome.test_not_palindrome) ... ok
test_simple_palindrome (__main__.TestPalindrome.test_simple_palindrome) ... ok

-----
Ran 3 tests in 0.002s

OK
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> |
```

Explanation :

- Tests are written first to define palindrome behavior.
- Cases include valid, invalid, and case-insensitive strings.

- The function is implemented only after tests.
- All tests pass, proving correct TDD usage.